

s-t Path with Consecutive Red Vertices

CS 6515 숙제 + Format Quiz 배경지식 · 핵심 포인트 정리 (핵심 용어는 영어로 표기)

Part I — 1. 문제 다시 읽기

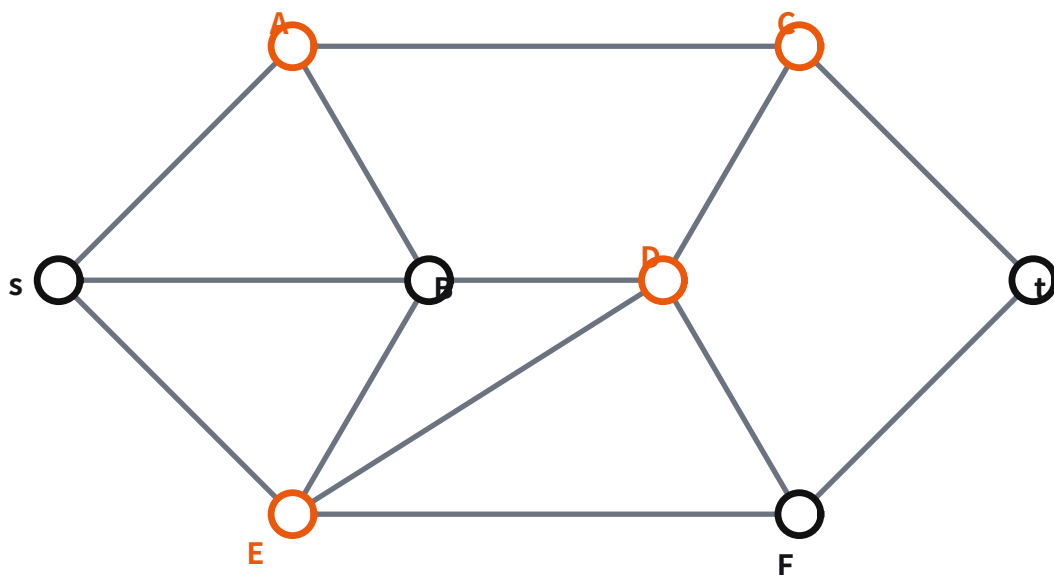
입력.

Undirected graph(무향 그래프) $G = (V, E)$. 모든 vertex는 red(빨강) 또는 black(검정)이며, 색은 $col(v)$ 로 $O(1)$ 시간에 조회 가능. 두 vertex s, t 가 주어지며 s 와 t 는 black이다.

판정.

s 에서 t 까지의 path(경로) 중, path가 지나는 red vertices가 모두 consecutively(연속해서) 등장하는 path가 존재하면 TRUE, 아니면 FALSE.

즉 유효한 path는 **black* · red+ · black***** 또는 **black*****(red 0개) 모양이어야 한다. Red가 두 덩어리로 나뉘면 무효다.



숙제 예제 그래프. 주황색 원 = red vertices(A, C, D, E), 검은 원 = black vertices(s, B, F, t).

Path 예시.

Valid ✓ $s-B-D-C-t \rightarrow$ red(D, C)가 한 덩어리.

Valid ✓ $s-A-C-D-F-t \rightarrow$ red(A, C, D)가 한 덩어리.

Invalid X $s-A-B-E-D-F-t \rightarrow$ red가 A와 (E, D) 두 덩어리로 분리 (path 자체는 존재하지만 조건 위반).

Part I — 2. 배경지식 A: graph, path, reachability

- **Undirected graph**: edge에 방향이 없어 $u-v$ edge는 양쪽으로 이동 가능하다.
- **Walk(워크)**: vertex를 반복해도 되는 이동 순서. **Path**: vertex를 반복하지 않는 walk. Reachability 문제에서는 walk가 존재하면 simple path(단순 경로)도 존재하므로(cycle을 잘라내면 된다) 둘을 구분하지 않아도 된다.

- **Reachability(도달 가능성):** “s에서 t로 갈 수 있는가?” — 이 문제의 최종 질문 형태다. 조건(“red 연속”)을 **그래프 구조로 번역**하면 나머지는 reachability 문제로 귀결된다.

Part I — 3. 배경지식 B: BFS/DFS는 reachability의 기본 도구

- BFS(breadth-first search, 너비 우선 탐색) / DFS(depth-first search, 깊이 우선 탐색)로 시작점에서 도달 가능한 모든 vertex를 $O(V + E)$ 시간에 찾을 수 있다.
- **Directed graph(방향 그래프)**에서도 BFS/DFS는 edge 방향을 따라가기만 하면 되므로 마찬가지로 $O(V + E)$ 이다. 아래의 layered graph는 방향성이 있으므로 기억해 두자.
- 핵심 통찰: 조건이 복잡해 보여도, **조건을 만족하는 이동만을 edge로 남긴 새 그래프**를 만들 수 있다면 문제는 “새 그래프에서 s에서 t에 reachability가 성립하는가?”가 된다.

Part I — 4. 배경지식 C: state augmentation

조건 “red vertices가 연속”은 path **전체**를 보고 판단해야 한다. BFS/DFS는 현재 vertex만 보므로, “지금 red 구간 안에 있는가?”라는 **memory(기억)**가 필요하다.

State augmentation(상태 확장)은 이 memory를 그래프 자체에 인코딩하는 기법이다. 각 vertex v 를 “상태가 다른 여러 복사본” (v , 상태)로 늘리고, 허용된 이동만 edge로 연결한다. 그러면 “조건을 만족하는 path” = “새 그래프에서의 (방향을 지키는) path”가 된다.

이 문제에서의 상태 3가지.

1. 아직 red에 들어가지 않음 — 지금까지 black만 지남
2. Red 구간 안에 있음 — 현재 red를 연속으로 지나는 중
3. Red 구간에서 나옴 — red 덩어리가 끝났고 다시 black만 지남

상태 전이는 $0 \rightarrow 1 \rightarrow 2$ 한 방향으로만 일어난다. 한 번 red 구간을 나오면(2) 다시 red(1)으로 돌아갈 수 없으므로, red는 자동으로 최대 하나의 연속 덩어리가 된다.

Part I — 5. 왜 naive한 방법으로는 안 되는가

Naive approach(단순한 시도)	Why it fails(왜 실패하는가)
Black vertex만 남기고 s-t 연결 확인	예제에서 black vertex만으로는 s-t path가 아예 없다(B의 black neighbor는 s뿐). Red를 꼭 써야 하는 경우를 놓친다.
Red vertex를 모두 지우고 확인	위와 같은 이유. Red 구간이 필요한 유효 path를 버리게 된다.
모든 s-t path를 열거해서 검사	Path의 수가 exponential(지수적)으로 많을 수 있어 polynomial time(다항 시간) 안에 안 끝난다.
DFS로 path 하나를 찾아 검사하고 실패하면 백트래킹	최악의 경우 exponential time. “Red 연속”은 전역 조건이라 greedy(탐욕적)으로 고른 path가 막혀도 다른 path가 유효할 수 있다.

즉 **red를 아예 배제할 수도, 무제한 허용할 수도 없고**, “정확히 하나의 연속 덩어리”라는 전역 조건을 polynomial time에 검사할 방법이 필요하다 → state augmentation이 정답 방향이다.

Part I — 6. 핵심 아이디어: 3개 layers

새 그래프 H 를 만든다. 각 vertex v 에 대해 상태별 복사본을 둔다:

- **Layer 0** ($v, 0$): red 진입 전 — **black vertex만** 존재
- **Layer 1** ($v, 1$): red 구간 안 — **red vertex만** 존재

• **Layer 2** ($v, 2$): red 탈출 후 — **black vertex**만 존재

Edge 규칙 (원본 edge $u-v$ 마다):

원본 edge 종류	H에 추가되는 edge	의미
black-black	$(u,0)-(v,0)$ undirected, $(u,2)-(v,2)$ undirected	Red 구간 전/후의 black 이동
black(u)-red(v)	$(u,0) \rightarrow (v,1)$ directed	Red 구간으로 진입 (되돌아갈 수 없음)
red-red	$(u,1)-(v,1)$ undirected	Red 구간 안에서의 이동
red(u)-black(v)	$(u,1) \rightarrow (v,2)$ directed	Red 구간에서 탈출 (되돌아갈 수 없음)

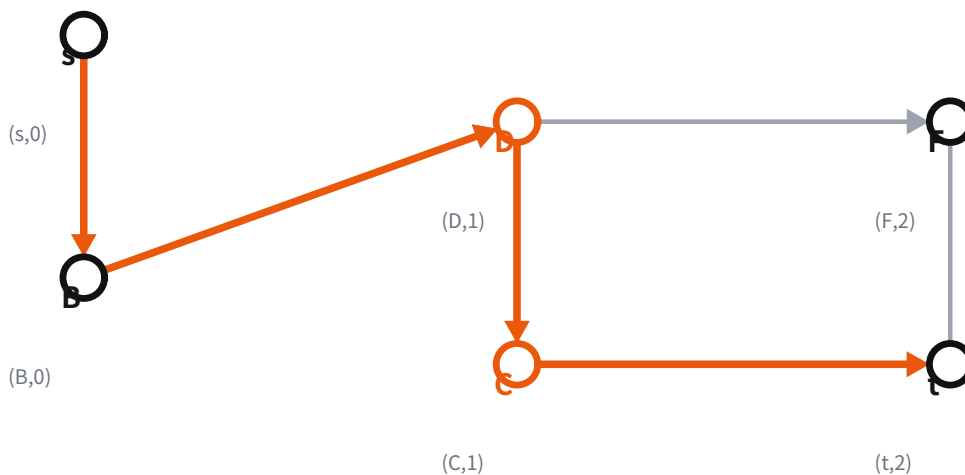
방향이 핵심이다. Layer 사이 edge를 directed로만 두어야 “ $0 \rightarrow 1 \rightarrow 2$ ” 순서가 강제되고, red(1층 vertices)가 path상 하나의 연속 블록이 된다. Layer 내부는 undirected graph이므로 양방향 이동 가능.

질의: ($s, 0$)에서 시작해 directed BFS/DFS로 (**$t, 0$**) 또는 (**$t, 2$**)에 도달 가능한가? ($t, 0$) 도달 = red 없는 유효 path, ($t, 2$) 도달 = red 덩어리를 포함한 유효 path.

Layer 0 · red 이전

Layer 1 · red 구간

Layer 2 · red 이후



예제 그래프의 일부를 layered graph로 펼친 모습. 주황색 화살표 = 유효 path $s-B-D-C-t$ 에 대응하는 walk $(s,0) \rightarrow (B,0) \rightarrow (D,1) \rightarrow (C,1) \rightarrow (t,2)$. Layer 간 이동은 directed라 red 구간으로 되돌아갈 수 없다.

Part I — 7. 예제 따라가기: $s-B-D-C-t$

1. ($s, 0$)에서 시작. s 는 black $\rightarrow$ layer 0.
2. $s-B$ 는 black-black edge $\rightarrow$ layer 0 내부 이동: (**$B, 0$**).
3. $B-D$ 는 black-red edge $\rightarrow$ directed 진입: (**$D, 1$**). 이제 red 구간 안.
4. $D-C$ 는 red-red edge $\rightarrow$ layer 1 내부 이동: (**$C, 1$**).
5. $C-t$ 는 red-black edge $\rightarrow$ directed 탈출: (**$t, 2$**). 도달!

Path상 red vertices D, C 는 layer 1에 머문 연속 구간에 정확히 대응한다. 반면 무효 path $s-A-B-E-D-F-t$ 는 ($A,1$) 진입 후 ($B,0$)으로 되돌아갈 edge가 없어 layered graph에서 재현되지 않는다.

손으로 확인.

$s-A-C-D-F-t$ 를 layered graph에서 따라가면 $(s,0) \rightarrow (A,1) \rightarrow (C,1) \rightarrow (D,1) \rightarrow (F,2) \rightarrow (t,2)$ 로 ($t,2$)에 도달한다. Red(A,C,D)가 layer 1에 연속으로 모여 있다.

Part I — 8. Correctness · Runtime 체크포인트

Ed #385 규칙에 따라 최종 답안은 **서술형 (a)/(b)/(c)**로 작성한다(의사코드 금지).

(a) Algorithm — 무엇을 만드는가

- 입력 G , 색 함수 $col, s, t(\text{black})$. Layered graph H 를 만든다: vertex 최대 $3V$ 개, edge는 원본 edge당 최대 3개 + layer 간 directed edge라 $O(V + E)$ 크기.
- $(s, 0)$ 에서 directed BFS/DFS를 돌려 $(t, 0)$ 또는 $(t, 2)$ reachability 여부를 반환.

(b) Correctness — 왜 맞는가 (두 방향)

- ($\Rightarrow$) 유효 path $P = \text{black}^* \cdot \text{red}^* \cdot \text{black}^*$ 가 존재하면, 각 vertex를 속한 구간의 layer로 들어올린 walk가 H 에 존재한다.
- ($\Leftarrow$) H 에서 $(s, 0) \rightarrow (t, 0)/(t, 2)$ walk가 존재하면, G 에 투영한 walk의 red vertices는 layer 1의 연속 구간에서만 나오므로 자동으로 연속이다. Walk는 cycle을 잘라 simple path로 만들 수 있다.

(c) Runtime analysis

- H 의 vertex $\leq 3n$, edge $O(m) \rightarrow$ 구축 $O(n + m)$.
- Directed BFS/DFS $O(V' + E') = O(n + m)$.
- $col(v)$ 는 $O(1)$. 총 $O(n + m)$.

Part I — 9. 함정 체크리스트

- **s, t 가 black**이라는 조건을 사용했는가? 시작 $(s, 0)$, 종료 $(t, 0)/(t, 2)$ 가 고정된다.
- **Red 0개**인 경우를 빠뜨리지 않았는가? $(t, 0)$ 도 확인해야 한다.
- Undirected edge 복사 시 **layer 내부는 undirected, layer 간은 directed**를 지켰는가?
- Layer 1에는 red만, layer 0/2에는 black만 두었는가?
- Correctness에서 “walk $\rightarrow$ path” 단계(cycle 제거 언급)를 빼먹지 않았는가?
- Runtime에서 H 의 크기가 원본의 상수 배임을 명시했는가?

Part I — 10. 확장 문제(UNGRADED) 힌트

s, t 가 임의의 색이어도 같은 layered graph를 쓴다. 달라지는 것은 시작/종료 layer뿐이다.

- **시작**: s 가 black이면 $(s, 0)$, red면 **$(s, 1)$** 에서 시작한다.
- **종료**: t 가 black이면 $(t, 0)$ 또는 $(t, 2)$. t 가 red면 **$(t, 1)$** 도달도 허용한다.
- s, t 모두 red이고 path 전체가 red면 layer 1 안에서만 이동해 $(t, 1)$ 에 도달한다.
- 핵심은 그대로: **조건(red 연속)은 layer 구조가 강제하고, 시작/종료 layer만 색에 맞게 고른다.**

Part I — 11. 손으로 해보기

- 예제 그래프를 종이에 그리고 $s-B-D-C-t$ 의 각 vertex 옆에 layer 번호(0/1/2)를 적어본다.
- 무효 path $s-A-B-E-D-F-t$ 가 어느 지점에서 layered graph를 따라갈 수 없는지 짚어본다.
- Vertex 4개짜리 작은 그래프를 직접 만들어 layered graph를 그리고 BFS를 손으로 돌려본다.
- 확장 문제: s 가 red인 작은 예제를 만들어 시작 layer를 $(s, 1)$ 로 두고 반복한다.

Part II — Format Quiz: "Avoiding x"

수업 Format Quiz on Graph Algorithms의 문제와 해설을 함께 공부하는 파트다. 이 퀴즈의 진짜 목적은 정답 맞추기가 아니라 **답안 형식** 훈련이다: **(a) algorithm / (b) correctness justification / (c) runtime analysis**.

퀴즈가 못 박은 규칙 3가지.

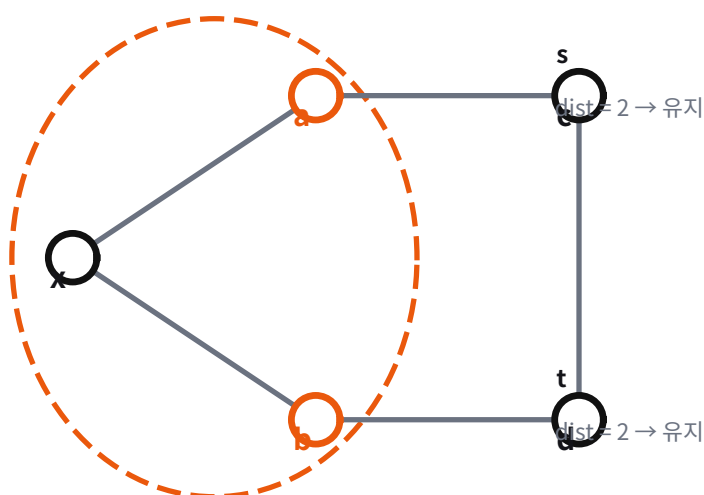
1. **Black box**는 수정하지 않는다. DFS, BFS, Dijkstra's 같은 black box는 그대로 쓰고, **입력(그래프)**을 변형해서 black box가 답을 찾게 만든다. Black box를 수정하면 동작해도 감점된다.
2. **(b) justification**에서는 알고리즘이 왜 동작하는지, 특히 **왜 입력을 그렇게 변형했는지** 이유를 설명해야 한다.
3. **(c) runtime**에서는 최종 값만 적지 말고 **계산 과정을 보여준다**. 알려진 black box의 runtime은 값만 진술해도 되지만, 입력 변형 비용 + 각 black box의 runtime을 항별로 합산해야 한다.

Q1. 문제 "Avoiding x"

입력.

Unweighted, undirected graph $G = (V, E)$ (adjacency list). Vertex x 와 값 k 가 주어진다. x 및 x 로부터 거리 $\leq k$ 인 모든 vertex를 피하면서, s 에서 t 까지의 **shortest path**를 찾는다. 그런 path가 존재한다고 가정한다.

배경지식. x 에서 BFS를 돌리면 모든 vertex v 에 대해 $\text{dist}[v] = x$ 로부터의 shortest path 거리를 얻는다. $\text{dist}[v] \leq k$ 인 vertices는 정확히 x 중심 반경 k 의 **ball** $B(x, k)$ 다. 이들을 제거하면 남은 그래프 G' 에서의 모든 s - t path는 자동으로 금지 vertices를 피한다.



$B(x, k=1)$: $\text{dist} \leq k \rightarrow$ 제거

x 에서 BFS $\rightarrow$ dist 계산. $\text{dist} \leq k$ 인 vertices(a, b)를 제거하면 남은 그래프에서 s - t shortest path(c - d)는 자동으로 조건을 만족한다.

Q2. 네 가지 후보 해법 a-d

(a)
 x 에서 BFS로 dist 계산 $\rightarrow$ *BFS를 수정*해서 $\text{dist} \leq k$ 인 vertex를 방문하려 할 때 스킵 $\rightarrow$ 수정된 BFS를 s 에서 실행해 shortest path 반환.
 $\rightarrow$ 동작은 맞지만 **black box(BFS) 내부**를 수정했으므로 코스 규율 위반. 감점 대상, full credit 아님.

(b)
 x 에서 BFS로 dist 계산 $\rightarrow$ $\text{dist} \leq k$ 인 vertices를 *그래프에서 제거* $\rightarrow G'$ 에서 s 부터 BFS $\rightarrow$ shortest path 반환.
 $\rightarrow$ Black box를 그대로 사용. Correct + $O(n+m)$. Optimal.

(c) — (b)의 pseudocode 버전

dist = BFS(G, x).dist → dist ≤ k인 vertex 제거 → BFS(G', s) → prev array를 t부터 거슬러 올라가며 path 복원.
→ Correct + optimal. Path 복원 과정까지 명시한 가장 완전한 형태.

```
dist = BFS(G, x).dist
G' = G
for v in V:
    if dist[v] <= k: G'.remove(v)
prev = BFS(G', s).prev
path_list = t; v = t
while prev[v] not null:
    v = prev[v]; path_list = v + path_list
return path_list
```

(d)

모든 edge cost가 1이라 가정 → x에서 Dijkstra's로 dist 계산 → dist ≤ k인 vertex에 닿는 edge들의 cost를 infinite로 변경 → G'에서 s부터 Dijkstra's.

→ Correct하지만 $O((n+m) \log n)$. Unweighted graph에서 Dijkstra's는 과하다 — BFS로 충분. Optimal 아님.

Q3. Q1 분석 — "correct and optimal" 고르기

- **Correctness:** 네 해법 모두 동작은 맞다 (“the above solutions are all correct in some way”).
- **Optimality:** 입력을 읽는 데만 $\Omega(n+m)$ 이 필요하므로 **$O(n+m)$ 이 최적.** (d)는 $O((n+m) \log n)$ 이라 탈락.
- **Black-box discipline:** (a)는 BFS를 수정했으므로 감점 → full credit 아님.
- **결론:** (b)와 (c)가 **correct and optimal**. Stimulus도 “pick any that would receive full credit”라 했다. (c)는 path 복원까지 포함한 가장 완전한 서술이다.

시험 포인트. “동작한다”와 “full credit을 받는다”는 다르다. Black box 수정과 불필요한 Dijkstra's 사용이 대표적인 감점 사유다.

Q4. Q2 매칭 — correctness justification

해법	매칭되는 정당성	왜
(a)	**6.** “By *modifying BFS* to avoid unwanted vertices, ...”	BFS 자체를 수정하는 해법(a)에 대응.
(b)	**3.** “By *modifying the graph* to remove any unwanted vertices, ...”	그래프를 변형하는 해법(b)에 대응. 정점을 제거했으므로 BFS가 찾는 모든 path는 자동으로 금지 vertices를 피한다.
(c)	**1.** “Our *code* removes any vertices ... follow the *prev array* from t to s ...”	코드로 쓴 (c)에 대응 (“Our code”, “prev array” 언급).
(d)	**4.** “... make edges ... *cost infinite*, *Dijkstra's* will avoid them ...”	Infinite cost + Dijkstra's를 쓰는 (d)에 대응.

Distractors: 2. “이미 설명했으니 정당성은 불필요” — Ed #385 정면 위반 (justification은 필수). 5. “TA가 된다고 했다” — 근거가 아님.

포인트: 좋은 justification은 “왜 입력을 변형했는가”를 설명한다. 변형 덕분에 black box의 출력이 곧 정답이 된다는 연결고리가 핵심 문장이다.

Q5. Q3 매칭 — runtime analysis

해법	매칭되는 런타임	왜
(a)	**R1.** 첫 BFS $O(n+m)$ + modified BFS도 $O(n+m) \rightarrow O(n+m)$	두 번의 BFS 언급.
(b)	**R2.** 첫 BFS $O(n+m)$ + 정점/간선 제거 $O(n+m)$ + 두 번째 BFS $O(n+m) \rightarrow O(n+m)$	제거 단계의 비용 명시.
(c)	**R3.** BFS(G, x) $O(n+m)$ + for v in V $O(n)$ + BFS(G', s) $O(n+m)$ + prev 추적 $O(n) = O(n+m)$	코드 한 줄 한 줄의 항별 계산.
(d)	**R4.** Dijkstra's 2회 $O((n+m) \log n)$ + edge 업데이트 $O(n+m) \rightarrow O((n+m) \log n)$	지배항이 Dijkstra's.

Distractors: R5. “ $O(nm)$ ” — 틀린 값. R6. “ $O(n+m)$ ”만 적음 — 계산 과정 없음 ((c) 규칙 위반). R7. “BFS가 $O(n+m)$ 이니까 최종도 $O(n+m)$ ” — 다른 단계들의 비용 무시, 불완전.

포인트: runtime은 **항별 합산**으로 쓴다. Black box는 알려진 값 진술로 충분하고, 입력 변형(제거 loop, edge 업데이트 등)의 비용은 직접 계산한다.

Q6. 핵심 포인트 + Part I(숙제)와의 연결

- “그래프 변형 + black box” 패턴. Part I 숙제(layered graph를 만들어 BFS/DFS)와 이 퀴즈(금지 vertices 제거 후 BFS)가 같은 패턴이다. GR 유닛의 핵심 테크닉: 조건을 그래프 구조로 번역한 뒤, 손대지 않은 black box를 돌린다.
- **Black box는 고치지 않는다.** (a)처럼 BFS 내부를 뜯는 대신, 입력 그래프를 바꾼다. Layered graph, 정점 제거, edge cost 변경이 모두 “입력 변형”의 예시다.
- **문제에 맞는 black box를 고른다.** Unweighted $\rightarrow$ BFS $O(n+m)$. Weighted $\rightarrow$ Dijkstra's $O((n+m) \log n)$. Unweighted에서 Dijkstra's를 쓰면 optimal을 놓친다.
- **Justification 공식:** “이렇게 입력을 변형했으므로, black box가 반환하는 것은 정확히 우리가 원하는 것이다.”
- **Runtime 공식:** $\Sigma(\text{변형 비용}) + \Sigma(\text{black box 비용})$, 항별로 명시.

학습용 노트이며, 제출 답안은 Ed #385에 따라 서술형 (a)/(b)/(c)로 직접 작성해야 한다. 퀴즈의 선택지·매칭은 이해를 위한 것이지, 답안에 그대로 옮겨 적는 형식이 아니다.